

Learn JavaScript: Beginner's Course

Lesson 5: Functions

Functions - Study Notes

Key Concepts

- **Function definition** – how to declare a function (named or anonymous).
- **Calling a function** – invoking the function with arguments.
- **Parameters & arguments** – placeholders in the definition vs. values passed when called.
- **Return values** – using `return` to send a result back to the caller.
- **Arrow functions** – concise syntax for writing functions (especially useful for callbacks).

Summary

A function is a reusable block of code that performs a specific task. You define a function once and can invoke (call) it many times, optionally passing data in the form of **parameters**. Inside the function you can manipulate those inputs and, if needed, **return** a result to the caller. Modern JavaScript (and many other languages) also offers **arrow functions**, a shorthand syntax that omits the `function` keyword and implicitly returns the expression when the body is a single statement. Understanding how to define, call, pass arguments, and retrieve return values is foundational for writing modular, maintainable code.

Important Points

- **Definition syntax** (traditional):

```
```js
function functionName(param1, param2) {
 // body
 return value;
}
```
```

- **Arrow function syntax** (concise):

```
```js
const functionName = (param1, param2) => {
 // body
 return value;
};
// If the body is a single expression, braces and return can be omitted:
const add = (a, b) => a + b;
```
```

- **Parameters** are variables listed in the function definition; **arguments** are the actual values supplied when the function is called.
- If no `return` statement is executed, the function implicitly returns `undefined` (in JavaScript).

- Functions can be **first class objects**: they can be assigned to variables, passed as arguments to other functions, and returned from functions.

- Arrow functions **do not bind their own** `this`, `arguments`, `super`, or `new.target`; they inherit `this` from the surrounding lexical scope.

Examples

Example 1 – Traditional function with parameters and return value

```
``js
// Calculates the area of a rectangle
function rectangleArea(width, height) {
  // Parameters: width, height
  return width * height; // Return the computed area
}
// Calling the function
let area = rectangleArea(5, 3); // Arguments: 5, 3
console.log(area);             // Output: 15
``
```

Explanation:

- `width` and `height` are parameters.
- When we call `rectangleArea(5, 3)`, the values `5` and `3` are the arguments.
- The function multiplies them and returns the product, which we store in `area`.

Example 2 – Arrow function used as a callback

```
``js
const numbers = [1, 2, 3, 4, 5];
// Arrow function that doubles a number
const doubled = numbers.map(n => n * 2);
console.log(doubled); // Output: [2, 4, 6, 8, 10]
``
```

Explanation:

- `map` expects a function that receives each array element.
- The arrow function `n => n * 2` is a concise way to write `function(n) { return n * 2; }`.
- Because the body is a single expression, the `return` keyword and braces are omitted.

Quick Review

1. What is the difference between a **parameter** and an **argument**?
2. How does an arrow function differ from a traditional function regarding the `this` keyword?
3. What value does a function return if it has no explicit `return` statement?
4. Write an arrow function that takes a string and returns its length.
5. True or False: Functions can be passed as arguments to other functions.

Further Reading

- **Higher order functions** – functions that accept or return other functions (e.g., `filter`, `reduce`).

- **Closures** – how functions retain access to variables from their lexical scope.
- **Default parameters** – providing fallback values for function arguments.
- **Rest parameters** (`...args`) – collecting an indefinite number of arguments into an array.
- **Generator functions** (`function*`) – functions that can pause execution and yield values lazily.

These notes cover the essentials of defining, calling, parameterizing, returning, and using arrow functions—core building blocks for any programmer.

